

Part 1: How Code Works (The Pipes & Filters Architecture)

Your Python script is built using the **Pipes and Filters** architectural pattern. It acts as a background pipeline that processes data sequentially in four distinct stages without ever slowing down the main Gateway.

Here is the exact breakdown of the code you wrote:

- **The Foundation (watchdog & __init__):** Instead of writing a clunky script that constantly opens and closes a text file in an infinite loop, you used the watchdog library. In the `__init__` function, your code uses `f.seek(0, os.SEEK_END)`. This means when your engine boots up, it skips all old logs and only waits for *new* traffic, saving massive amounts of RAM and CPU.
- **Filter 1: The Listener (process_new_logs):** When the watchdog detects a change in the log file, this function wakes up, reads only the newly appended lines, and pipes them directly to Filter 2.
- **Filter 2: The Parser (filter2_parse):** This is where you extract data. You used a Regular Expression `(r'^(\d+\.\d+\.\d+\.\d+).*\s(\d{3})\s')` to instantly scan the messy raw text log and pull out two specific variables: the **Client IP Address** and the **HTTP Status Code**. You programmed it to ignore successful traffic (200 OK) and only pipe forward malicious/error codes (401, 403, 404, 500).
- **Filter 3: The Detector (filter3_detect):** This is your algorithmic masterpiece—the sliding window.
 - Every time an error occurs, you log the exact timestamp (`time.time()`).

- Using Python list comprehension, your code dynamically scrubs the list, deleting any timestamps older than `TIME_WINDOW_SEC` (10 seconds).
- If the length of that list ever exceeds `ERROR_THRESHOLD` (50 errors), it triggers a breach and pipes the IP to Filter 4.
- **Filter 4: The Blocker** (`filter4_block`): This function connects to Redis. It checks if the IP is already blocked (`r.sismember`). If it isn't, it immediately adds the IP to the Redis blacklist (`r.sadd("blocked_ips", ip)`) and prints a threat alert to the console.

2 How Code Relates to the Existing Files

- **Relationship with** `docker-compose.yml`:
 - In your Python code, you used `os.getenv("REDIS_HOST")` and `os.getenv("LOG_FILE_PATH")`.
 - If you look at the `docker-compose.yml` file under the `anomaly-engine` service, your team defined `REDIS_HOST=redis` and `LOG_FILE_PATH=/var/log/shieldapi/access.log`. Your Python code dynamically pulls these exact values to know where the database and log files live.
- **Relationship with** `Dockerfile.txt`:
 - The `Dockerfile.txt` for your engine ends with the command `CMD ["python", "-m", "engine.main"]`. This is why renaming your file to `main.py` was critical. When Docker finishes building the Linux container, it fires that exact command to boot up your pipeline.

- **Relationship with requirements.txt:**
 - Your script relies on import redis and from watchdog.observers import Observer. The requirements.txt file guarantees that redis>=5.0.0 and watchdog>=4.0.0 are installed inside the Docker container before your script runs.

Part 3: How You Integrate with Your Teammates

The integration between your microservice and your friends' microservices relies on two critical "Bridges": **Shared Storage** and **Shared Memory**.

1. Integration with the Gateway (The Shared Storage Bridge)

You and the teammate building the FastAPI Gateway are entirely decoupled, meaning your code never directly talks to their code.

- **How it connects:** In the docker-compose.yml, both the gateway and the anomaly-engine are mounted to the exact same volume: - shared_logs:/var/log/shieldapi.
- **The Workflow:** The Gateway handles a user request and writes a line of text to /var/log/shieldapi/access.log. Because you share that exact folder via Docker, your watchdog script instantly "hears" that write operation and processes the log.

2. Integration with Redis & Gateway (The Shared Memory Bridge)

Once your engine detects a threat, it has to tell the Gateway to stop letting the attacker in.

- **How it connects:** Both your Anomaly Engine and the Gateway are connected to the exact same Redis container on port 6379.
- **The Workflow:** Your Filter 4 executes `r.sadd("blocked_ips", ip)`. The moment you write that IP into Redis, the Gateway (which checks Redis on every single incoming request) instantly sees the IP is banned and drops the attacker's connection with a 403 Forbidden error.

3. Integration with the Dashboard (The Visual Bridge)

- **How it connects:** The React Dashboard also reads from that exact same Redis broker.
- **The Workflow:** The moment your Anomaly Engine writes a banned IP to Redis, the Dashboard's API fetches it and displays the blocked IP on the UI's Anomaly Feed in real-time.